

MODULE 01

선형회귀, AI가 직선을 그리는 법

점들을 보고 AI가 스스로 직선 $y = Wx + b$ 를 찾아내는 원리 —
경사하강법으로 배우는 머신러닝의 첫걸음

🕒 약 30분

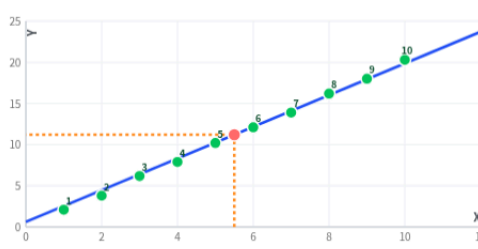
🎒 준비물 없음

📖 선수 지식: 일차함수 $y = ax + b$

학습 순서: ① 비유로 이해하기 (지금) → ② 시뮬레이터로 체험하기 → ③ 파이썬 코드로 확인하기

평면좌표 데이터 점 (10/10개)

점을 찍어보세요 — AI 공부의 첫걸음이 예요 🌱 [+ 10개 점 자동제 우기](#)



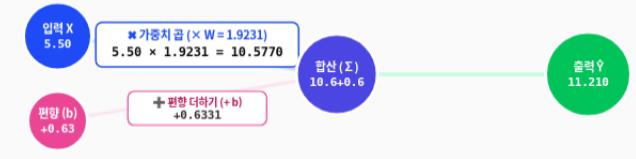
📌 캔버스를 클릭하면 점이 추가됩니다. (우측 슬라이더 조절 시 회귀선이 실시간 연동)

No	입력 X	출력 Y	삭제
1	1	2.1	삭제
2	2	3.8	삭제

✅ 체크포인트: 점 10개가 채워지면 다음 카드로 고!

단일 뉴런 순전파: 동그란 노드를 통한 가중치(W) 곱 전달

신호가 동그란 노드를 타고 흘러가요 [+ 신호 재생](#) [연속 전달](#)



✅ 체크포인트: 1 → 2 → 3 → 4단계 순서로 숫자가 흘러가면 성공!

1~4단계 실시간 파라미터 조절 (임의 조절 시 노드에 즉시 반영)

마음껏 움직여보세요 — AI는 실패해도 괜찮아요 😊 슬라이더를 드래그하면 바로 반영돼요

1단계

입력 노드 (X)

x = 5.5000 입력값

x = 5.5000

2단계

가중치 곱 (x × W)

W = 1.9231 가중치(W)

5.50 × 1.9231 = 10.5770

3단계

편향 더하기 (+ b)

4단계

출력 노드 (Ŷ)

▲ 선형회귀 & 단일 뉴런 가중치 전파 시뮬레이터 — 점을 찍으면 AI가 직선을 찾아냅니다

START · 이 모듈의 지도

30분 뒤, 여러분은 AI의 공부법을 설명할 수 있습니다

이 모듈은 머신러닝의 가장 기본이 되는 원리를 다룹니다. 어렵게 느껴질 수 있지만, 사실 여러분이 중학교 때 배운 일차함수 $y = ax + b$ 가 전부입니다. AI는 이 직선을 '스스로' 찾아낼 뿐이에요.

학습 순서 — 3단계

1 비유로 이해하기

가중치·편향·손실·경사하강법을 일상 비유로 먼저 이해합니다. (지금 이 페이지부터!)

2 시뮬레이터로 체험하기

마우스 클릭과 슬라이더로 AI가 직선을 찾아가는 과정을 직접 눈으로 봅니다.

3 파이썬 코드로 확인하기

체험한 내용을 그대로 옮긴 파이썬 코드를 한 줄씩 실행하며 확인합니다.

이 모듈에서 배우는 것

- ① 점들을 보고 AI가 스스로 직선($y = Wx + b$)을 찾아내는 원리
- ② 경사하강법 — AI가 조금씩 정답에 가까워지는 공부 방법
- ③ 단일 뉴런의 순전파 — 입력이 출력이 되기까지의 4단계

준비 완료 체크

- 일차함수 $y = ax + b$ 에서 a (기울기), b (y절편)가 무엇인지 기억한다.
- 시뮬레이터 페이지와 Colab 노트북(`colab_linear_regression.ipynb`)을 열어 두었다.

도입 · 질문

AI는 어떻게 점들 사이로 가장 잘 맞는 직선을 그을까?

시험 공부 시간(x)과 시험 점수(y)의 관계 — AI라면 이 점들 사이로 어떤 직선을 그을까요?

위 그래프의 점 10개를 보세요. 공부 시간이 늘수록 점수가 오르는 경향이 보이죠? 사람이라면 자를 대고 대충 직선을 그을 겁니다. AI도 똑같이 직선을 찾지만, 방식이 다릅니다. AI는 처음에 엉뚱한 직선을 그어 놓고, '틀린 정도'를 재면서 조금씩 고쳐 나갑니다. 이 '틀리면서 배우기'가 바로 머신러닝의 핵심입니다.

3줄 요약

- ① AI가 찾는 직선의 정체는 일차함수 $y = Wx + b$ 입니다. (W: 기울기, b: y절편)
- ② AI는 예측이 정답과 얼마나 다른지 **손실(MSE)**이라는 점수로 잽니다.
- ③ **경사하강법**으로 손실을 조금씩 줄이며 최고의 직선에 다가갑니다.

모듈 1 · 머신러닝: 선형회귀 — AI가 직선을 그리는 법

약 30분 · 준비물: 이 페이지 하나

이 모듈에서 배우는 것

- ① 점들을 보고 AI가 스스로 직선($y = Wx + b$)을 찾아내는 원리
- ② **경사하강법** — AI가 조금씩 정답에 가까워지는 공부 방법
- ③ 아래 시뮬레이터로 체험하고, 맨 아래 파이썬 코드로 직접 확인하기

준비물: 없음 (마우스 클릭만 하면 돼요!)

선수 지식: 일차함수 $y = ax + b$

비유로 먼저 이해해요!

- **가중치(W)**란? AI가 "이 입력이 얼마나 중요할까?"라고 생각하는 정도예요. (직선의 기울기)
- **편향(b)**이란? 입력이 0이어도 나오는 기본값이에요. (직선의 y절편)
- **손실(MSE)**이란? AI의 예측이 정답과 얼마나 다른지 재는 '틀린 정도 점수'예요.
- **경사하강법**이란? '틀린 정도'를 조금씩 줄여나가는 AI의 공부법이에요. **안개 낀 산에서 가장 낮은 골짜기를 찾아 내려가는 것과 같아요!**

▲ 모듈 1 도입 — 배울 내용 3줄 요약과 학습 목표

개념 ①

가중치(W) — AI의 '중요도' 판단

가중치(Weight, W)는 AI가 "이 입력이 얼마나 중요할까?"라고 생각하는 정도입니다. 일차함수 $y = ax + b$ 의 기울기 a 와 정확히 같은 역할을 합니다.

가중치(W)

입력 x 에 곱해지는 숫자. 직선의 **기울기**입니다. W 가 크면 x 가 조금만 변해도 출력이 크게 바뀝니다.

기울기의 의미

공부 시간(x)이 1시간 늘 때 점수(y)가 몇 점 오르는지를 나타냅니다. 시뮬레이터의 최종 답은 $W \approx 1.92$ 였습니다.

💡 비유 — 시험 점수

공부 시간의 가중치가 1.92라는 건 '1시간 더 공부하면 약 1.9점 오른다'는 뜻이에요. 반면 '어제 본 TV 시간'의 가중치는 0에 가깝겠죠. AI는 수많은 시행착오 끝에 각 입력의 진짜 중요도를 숫자로 찾아냅니다.

🔍 시뮬레이터 연결

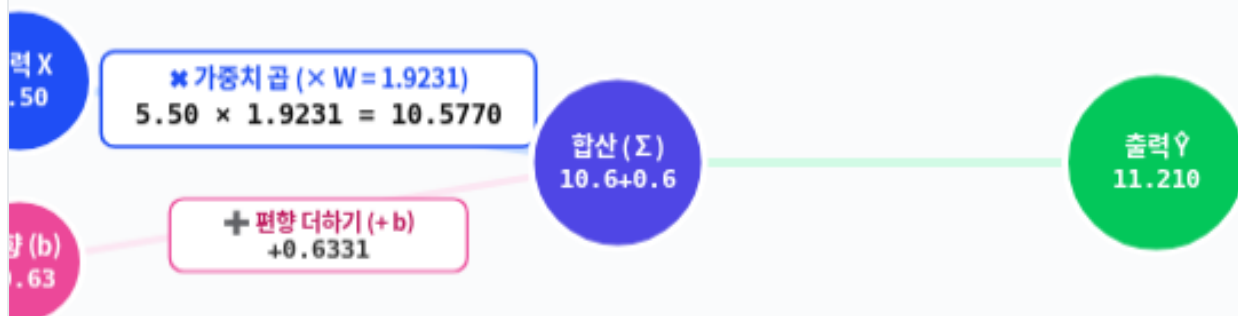
시뮬레이터의 1~4단계 실시간 파라미터 조절에서 W 슬라이더를 움직여 보세요. W 를 키우면 직선이 가팔라지고, 줄이면 완만해집니다. 기울기가 곧 AI의 판단임을 눈으로 확인할 수 있습니다.

순전파: 동그란 노드를 통한 가중치(W) 곱

신호가 동그란 노드를 타고 흘러가요 ✨

+ 신호 재생

☒ 연속 전달



1 → 2 → 3 → 4단계 순서로 숫자가 흘러가면 성공!

▲ 단일 뉴런 순전파: 입력 x 에 가중치 W 를 곱하는 2단계

개념 ②

편향(b)과 손실(MSE) — 기본값과 '틀린 정도' 점수

편향(Bias, b)

입력 x가 0이어도 나오는 기본값입니다. 일차함수의 y절편과 같습니다. 시뮬레이터의 최종 답은 $b \approx 0.63$ — 공부 시간 0이어도 기본 점수 0.63점이라는 의미죠.

손실(Loss, MSE)

AI의 예측이 정답과 얼마나 다른지를 재는 '틀린 정도' 점수입니다. MSE는 Mean Squared Error, 즉 (정답 - 예측)²의 평균입니다.

$$MSE = (1/n) \times \sum (\text{정답 } y - \text{예측 } \hat{y})^2$$

n개의 데이터에 대해 '얼마나 틀렸는지'를 제공해서 평균낸 값

🤔 왜 굳이 '제공'을 할까요?

- ① 부호 문제 해결: +3점 틀린 것과 -3점 틀린 것은 똑같이 나뉩니다. 제공하면 둘 다 9가 되어 방향과 상관없이 '틀린 크기'만 잴 수 있어요.
- ② 큰 실수에 엄격: 1점 틀리면 1점, 10점 틀리면 100점이 됩니다. 크게 틀릴수록 훨씬 무거운 벌점을 주어 AI가 빨리 고치게 만듭니다.

시뮬레이터 상단에는 수식 $y = 1.9231x + 0.6331$ | 손실(MSE): 0.0000이 표시됩니다. 학습이 끝난 AI가 찾아낸 최고의 직선, 그리고 그 직선의 틀린 정도 점수가 거의 0이라는 뜻입니다.



▲ 경사하강법 제어 패널 — 손실(MSE)이 줄어드는 과정을 실시간으로 봅니다

개념 ③

경사하강법 — 안개 낀 산을 내려가는 AI의 공부법

경사하강법(Gradient Descent)은 '틀린 정도(손실)'를 조금씩 줄여나가는 AI의 공부법입니다. 대표적인 비유는 안개 낀 산에서 가장 낮은 골짜기를 찾아 내려가는 것과 같습니다.

💡 비유 — 안개 낀 산

눈앞이 안 보이는 산에 서 있다고 상상해 보세요. 발밑의 기울기를 느껴 **가장 가파르게 내려가는 방향**으로 한 걸음씩 이동합니다. 한 걸음의 크기가 바로 **학습률(α)**이에요. 걸음이 너무 크면 골짜기를 지나쳐 반대편 산으로 뛰어오르고, 너무 작으면 밤을 새워도 못 내려갑니다. 이 과정을 **에포크(Epoch)** — 전체 데이터를 한 번씩 훑는 단위 — 만큼 반복합니다.

학습률(α)

한 번에 고치는 양. 시뮬레이터에서 직접 조절하며 '발걸음 크기'의 효과를 실험할 수 있습니다.

에포크(Epoch)

전체 데이터를 한 바퀴 학습하는 단위. 에포크가 쌓일수록 손실이 줄어듭니다.

기울기(Gradient)

어느 방향으로 얼마나 고쳐야 할지를 알려주는 나침반. 미분으로 계산합니다.

👉 교사용 팁 — 수업 운영

학습률 슬라이더를 극단적으로 크게(예: 1.0) 설정하고 '실시간 학습'을 눌러 보게 하세요. 손실이 요동치며 발산하는 모습을 보면 '적절한 학습률'의 의미가 몸으로 와닿습니다.

체험 ① · 시뮬레이터

점을 찍어보세요 — AI 공부의 첫걸음

시뮬레이터의 첫 화면은 **평면좌표 데이터 점** 캔버스입니다. AI에게 보여줄 '문제집'을 직접 만드는 단계예요.

1 점 찍기

캔버스를 클릭하면 점이 하나씩 추가됩니다. 공부 시간(x)과 시험 점수(y)를 상상하며 찍어 보세요.

2 자동 채우기

+ 10개 점 자동채우기 버튼으로 예제 데이터를 바로 불러올 수 있습니다.

3 데이터 확인

데이터 목록 표에서 X, Y 값을 직접 수정할 수도 있고, CSV로 다운로드할 수도 있습니다.

✅ 여기까지 되면 ✅

- 점 10개가 캔버스에 채워졌다.

학습 순서: ① 비유로 이해하기 (지금!) → ② 시뮬레이터로 체험하기 → ③ 파이썬 코드로 확인하기

평면좌표 데이터 점 (10/10개)

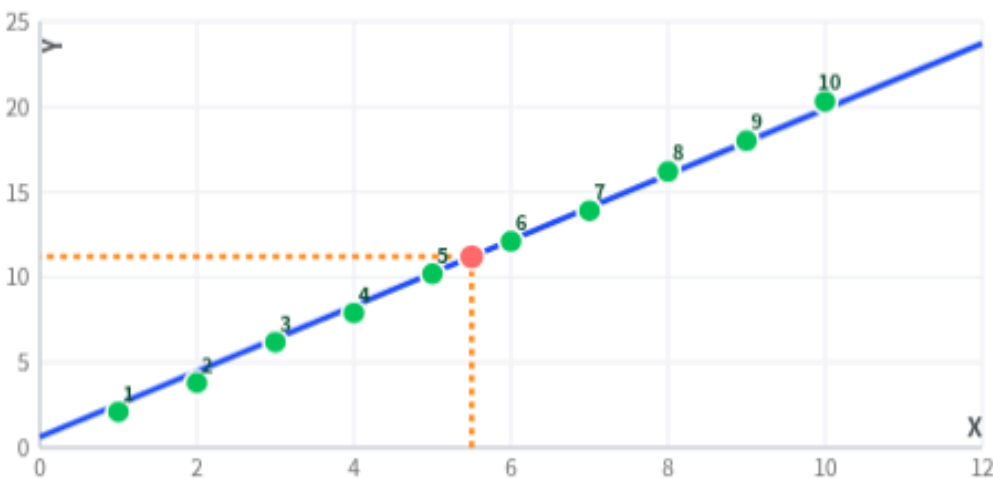
점을 찍어보세요 — AI 공부의 첫걸음
예요 🌱

+ 10개 점 자동채우기

단일
전달

✅ 체크포인트

외부 강의용 교재
1~4



💡 캔버스를 클릭하면 점이 추가됩니다. (우측 슬라이더 조절 시 회귀선이 실시간 연동)

데이터 목록 (수정 가능)

CSV 다운로드

초기화

▲ 데이터 점 입력 화면 — 클릭으로 점을 찍고 10개를 채웁니다

체험 ② · 시뮬레이터

공이 골짜기로 데굴데굴 — 실시간 학습

경사하강법 제어 & 2차 곡선 카드에서는 AI가 공부하는 모습을 실시간 애니메이션으로 봅니다. 2차 함수 손실 곡선 위에서 공이 데굴데굴 굴러가며 가장 낮은 지점을 찾아갑니다.

1 조건 설정

학습률(α), 에포크 수, 초기 W , 초기 b 를 슬라이더로 정합니다.

2 실시간 학습

▶ 실시간 학습 버튼을 누르면 공이 곡선을 따라 내려가며 에포크별 기록표가 채워집니다.

3 기록 확인

표의 손실(MSE) 열이 점점 줄어드는지 확인합니다. ⚡ 고속 학습으로 빠르게도 볼 수 있습니다.

✅ 여기까지 되면 ✅

- '실시간 학습'을 눌렀을 때 손실(MSE) 숫자가 점점 줄어든다.
- 에포크별 수렴 기록 표에 W , b , 손실이 한 줄씩 기록된다.



▲ 경사하강법 실시간 학습 — 2차 곡선 위를 공이 굴러 내려가며 손실이 감소합니다

체험 ③ · 시뮬레이터

단일 뉴런 순전파 — 신호가 흘러가는 4단계

이제 AI의 '뇌세포' 하나를 들여다봅니다. **단일 뉴런**은 입력이 출력이 되기까지 **4단계**를 거칩니다. + 신호 재생 버튼을 누르면 숫자가 1 → 2 → 3 → 4단계로 흘러가는 애니메이션을 볼 수 있습니다.

1 1단계 · 입력 노드

입력값 x 가 들어옵니다. (예: $x = 5.5$)

2 2단계 · 가중치 곱

$x \times W$ 를 계산합니다. (예: $5.50 \times 1.9231 = 10.5769$)

3 3단계 · 편향 더하기

$+ b$ 를 더합니다. (예: $+ 0.6331$)

4 4단계 · 출력 노드

최종 예측 $\hat{Y} = 11.2099$ 가 나옵니다.

✅ 여기까지 되면 ✅

- 1 → 2 → 3 → 4단계 순서로 숫자가 흘러가는 것을 확인했다.
- 슬라이더를 움직이면 노드의 숫자가 즉시 바뀌는 것을 확인했다.

▲ 단일 뉴런 순전파 — 4단계를 거쳐 예측값 $\hat{Y}$ 이 계산됩니다

코드 ① · 파이썬

체험을 코드로 옮기기 — 데이터와 초기값

시뮬레이터에서 마우스로 하던 일을 **파이썬 코드**로 그대로 옮기면 아래와 같습니다. Colab 노트북 (colab_linear_regression.ipynb)의 셀들과 같은 내용이니 한 줄씩 실행하며 비교해 보세요.

```
# 1. 10개의 샘플 데이터 준비 (X: 공부 시간, Y: 시험 점수)
# → AI에게 보여줄 '문제집' 10개예요.
X = np.array([1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0])
Y = np.array([2.1, 3.8, 6.2, 7.9, 10.2, 12.1, 13.9, 16.2, 18.0, 20.3])

# 2. 초기 가중치(W)와 편향(b) 설정
# → W(기울기)와 b(y절편)를 0.5에서 시작해요. 학습률(한 번에 고치는 양)은 0.01!
W = 0.5
b = 0.5
learning_rate = 0.01
epochs = 100
```

 colab_linear_regression.ipynb — 셀 2·3

코드 읽기 포인트

- ① **데이터(X, Y)**: 시뮬레이터에서 마우스로 찍던 점 10개와 같은 것입니다. 공부 시간 1~10시간에 대응하는 시험 점수예요.
- ② **초기값 $W = 0.5$, $b = 0.5$** : AI는 정답을 모른 채 **영똥한 직선부터** 시작합니다. 여기서부터 조금씩 고쳐 나가 는 거예요.
- ③ **learning_rate = 0.01**: 안개 낀 산 비유의 '한 걸음 크기'입니다.

코드 ② · 파이썬

AI의 공부 시간 — 경사하강법 반복문

이 모듈의 심장입니다. 아래 반복문 1바퀴가 'AI가 한 번 공부하는 과정' 전체예요.

```
# 3. 경사하강법 훈련 반복문: AI의 공부 시간! (안개 낀 산 내려가기 🏔️)
for epoch in range(1, epochs + 1):
    # [1~4단계 순전파] ① 지금 실력으로 점수 맞혀보기
    Y_pred = X * W + b

    # ② '틀린 정도 점수' 재기 (실제 점수와 예측 점수 차이의 제곱 평균)
    loss = np.mean((Y - Y_pred) ** 2)

    # ③ W와 b를 어느 방향으로 얼마나 고칠지 계산 (미분)
    w_gradient = -2 * np.mean(X * (Y - Y_pred))
    b_gradient = -2 * np.mean(Y - Y_pred)

    # ④ 기울기의 반대 방향으로 조금씩 이동! (산 내려가기)
    W -= learning_rate * w_gradient
    b -= learning_rate * b_gradient

    if epoch % 20 == 0:
        print(f"Epoch {epoch:3d} | W: {W:.4f} | b: {b:.4f} | MSE Loss: {loss:.4f}")
```

colab_linear_regression.ipynb — 셀 4

한 바퀴 = 4단계

- | | |
|---------|--|
| ① 예측 | 현재 W, b로 $Y_{pred} = X \times W + b$ 를 계산 — '지금 실력으로 맞혀보기' |
| ② 채점 | $loss = \text{평균}((Y - Y_{pred})^2)$ — '틀린 정도 점수' 재기 |
| ③ 방향 탐색 | 기울기(gradient) 계산 — W와 b를 어느 쪽으로 고칠지 정하기 |
| ④ 이동 | $W -= \text{학습률} \times \text{기울기}$ — 산을 한 걸음 내려가기 |

✅ 여기까지 되면 ✅

- 출력된 MSE Loss 숫자가 에포크마다 점점 줄어드는 것을 확인했다.

코드 ③ · 파이썬

최종 수식 완성과 시각화

100 에포크의 공부가 끝나면, AI가 찾아낸 최고의 직선을 발표합니다.

```
# 4. 최종 결과 출력 및 수식 완성
# → 학습이 끝난 AI가 찾아낸 직선의 수식!
print(f"\n최종 회귀식: y = {W:.4f} * x + {b:.4f}")

# 5. 시각화 (산점도와 회귀선)
# → 점들 사이에 AI가 그은 직선을 그려봐요
plt.scatter(X, Y, color='#1f4ef5', label='실제 데이터 (x, y)')
plt.plot(X, W * X + b, color='#03c75a', lw=2.5,
         label=f'예측선: y={W:.2f}x+{b:.2f}')
plt.xlabel('x (입력 노드)'); plt.ylabel('y (출력 노드)')
plt.title('선형회귀 가중치 전파 학습 결과')
plt.legend(); plt.grid(True, alpha=0.3); plt.show()
```

 colab_linear_regression.ipynb — 셀 5·6

최종 답

$$y = 1.9231x + 0.6331$$

학습이 끝난 AI가 찾아낸 직선 — 시뮬레이터 상단에도 표시됩니다

파란 점(실제 데이터)들 사이로 초록 직선(예측선)이 지나갑니다. 이 직선이 바로 AI가 100번의 시행착오 끝에 찾아낸 답입니다.

심화 탐구

학습률을 바꾸면 어떻게 될까? — 직접 실험하기

이제 여러분이 연구자가 되어 **하이퍼파라미터 실험**을 해 봅시다. 시뮬레이터의 슬라이더와 노트북의 변수 값을 바꿔 가며 관찰해 보세요.

실험 1 — 학습률(α) 바꾸기

학습률	예상 결과	관찰 포인트
0.001 (아주 작게)	손실이 아주 천천히 줄어듦	100 에포크 안에 골짜기에 못 닿는다
0.01 (기본값)	손실이 안정적으로 감소	공이 부드럽게 굴러 내려간다
0.5 (크게)	손실이 요동치며 발산	골짜기를 지나쳐 반대편으로 튕다

실험 2 — 초기 W , b 바꾸기

초기 직선을 엉뚱하게(예: $W = -2$, $b = 10$) 그어 놓고 학습시켜 보세요. 어디서 시작하든 결국 같은 최고의 직선에 수렴한다면, 경사하강법이 얼마나 든든한 방법인지 실감할 수 있습니다.

실험 3 — 데이터에 노이즈 섞기

점들을 일부러 삐뚤빼뚤 찍어 보세요. AI가 완벽한 직선이 아닌 '**가장 덜 틀리는**' 직선을 찾는 모습에서 머신러닝이 '평균적으로 잘 맞추는 법'임을 이해할 수 있습니다.

교사용 팁 — 토론 주제

'AI는 정답을 아는 게 아니라, 틀린 정도를 줄이는 중이다' — 이 문장이 맞는지 친구와 토론해 보세요.

세상 속 AI

직선 하나로 세상을 예측하는 AI들

선형회귀는 교과서 속 수식이 아니라, 지금 이 순간에도 세계 곳곳에서 돌아가고 있는 **실전 알고리즘**입니다.

집값 예측

부동산 AI는 **평수(x)**와 **집값(y)**의 관계에서 직선을 찾습니다. '평수가 1평 늘면 집값이 몇만 원 오르는가'가 바로 가중치 W 입니다.

수요 예측

편의점 본사는 **기온(x)**과 **아이스크림 판매량(y)**의 직선을 그려 내일 발주량을 정합니다. 여러분의 동네 편의점 진열대 뒤에도 선형회귀가 숨어 있습니다.

건강 지표

병원에서는 **나이(x)**와 **혈압(y)**의 관계식을 구해 건강 검진 기준을 만듭니다. 단, 여기서 중요한 경고가 있습니다.

⚠ 주의 — 상관관계와 인과관계: 직선이 잘 맞다고 해서 x 가 y 의 '원인'은 아닙니다. 아이스크림 판매량과 익사 사고는 함께 오르지만, 둘 다 더워 때문이지 서로의 원인이 아닙니다. AI의 예측을 해석할 때는 항상 '그래서 진짜 원인이 뭘까?'를 물어야 합니다.

Q&A

자주 묻는 질문 — 한 걸음 더 깊이

? AI가 정말 '생각'하면서 직선을 찾는 건가요?

아닙니다. AI는 기율기라는 **수학적 나침반**을 따라 기계적으로 이동할 뿐입니다. '생각'이 아니라 '계산'이죠. 하지만 이 단순한 계산을 수백만 번 반복하면 사람 눈에는 생각처럼 보일 만큼 정교한 결과가 나옵니다. 이 모듈에서 배운 경사하강법이 바로 그 계산의 정체입니다.

? 왜 하필 '직선'인가요? 곡선은 안 되나요?

좋은 질문입니다! 직선으로 부족하면 **곡선**을 쓰면 됩니다. 시뮬레이터에도 '2차 곡선' 옵션이 있죠. 다만 곡선은 구부러지는 정도를 조절하는 항이 늘어나 계산이 복잡해집니다. 머신러닝의 원칙은 '**가장 단순한 것부터**' — 직선으로 설명되면 직선을 씁니다.

? 에포크 100번이면 충분한가요? 더 하면 더 좋아지나요?

어느 지점을 넘기면 손실이 거의 줄지 않는 **수렴** 상태에 도달합니다. 그 이후로 계속 학습시키면 오히려 **주어진 점들에만 지나치게 맞추는(과적합)** 문제가 생길 수 있어요. '적당히'가 핵심이라는 것도 AI 학습의 중요한 교훈입니다.

? 사람도 이렇게 배우나요?

비슷합니다! 여러분이 수학 문제를 틀리고 오답노트를 정리하는 과정도 '틀린 정도를 줄이는' 과정이죠. 다만 사람은 한두 번의 실수로도 요령을 터득하지만, AI는 수천 번의 반복이 필요합니다. 대신 AI는 지치지 않고, 한 번 배운 것을 정확히 기억합니다.

정리 · 체크리스트

여기까지 되면 — 모듈 1 완주!

30분의 여정이 끝났습니다. 아래 항목을 하나씩 확인해 보세요. 모두 체크했다면 여러분은 이미 머신러닝의 핵심 원리를 손에 쥔 겁니다.

 핵심 개념 체크

- **가중치(w)**가 기울기, 즉 '입력의 중요도'라는 것을 설명할 수 있다.
- **편향(b)**이 y절편, 즉 '기본값'이라는 것을 설명할 수 있다.
- **손실(MSE)**이 (정답-예측)²의 평균인 '틀린 정도 점수'임을 안다.
- **경사하강법**을 '안개 낀 산 내려가기' 비유로 설명할 수 있다.
- **학습률**이 너무 크면 발산하고, 너무 작으면 느리다는 것을 안다.

 체험·코드 체크

- 시뮬레이터에서 점 10개를 찍고 실시간 학습으로 손실이 줄어드는 것을 보았다.
- 단일 뉴런의 1→2→3→4단계 순전파를 따라가 보았다.
- 파이썬 코드의 for 반복문이 ①예측→②채점→③방향→④이동임을 안다.

 한 문장 정리

선형회귀 = $y = Wx + b$ 라는 직선을, 틀린 정도(MSE)를 재면서 조금씩(경사하강법) 찾아내는 것.

 교사용 팁 — 다음 차시 예고

다음 모듈에서는 '숫자'로 세상을 보는 AI의 눈 — **컴퓨터 비전**을 배웁니다. 사진이 사실은 숫자 행렬이라는 놀라운 이야기부터 시작합니다.